REVIO STOCKSENSE

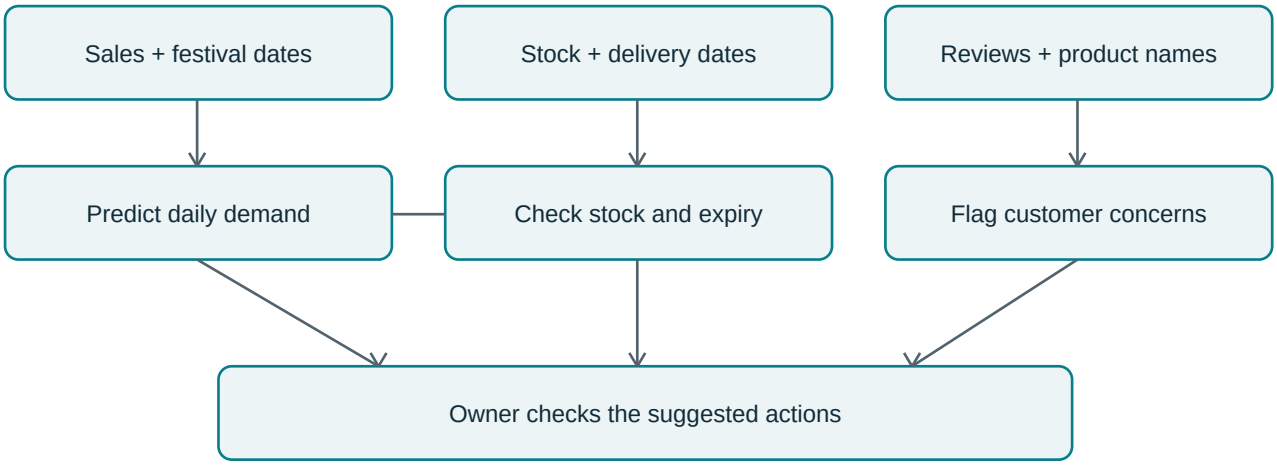
Technical Approach Report

The model, the dataset, the error scores and the stock calculations.

17 October 2026
Independent proposal for Cynbit AI/ML Engineer Internship

The shop data is generated by a program. Real-shop accuracy and savings are not yet known. The app does not connect to Revio or place orders.

System at a glance



How the current application works, explained in everyday language. The model and dataset were not changed for this rewrite.

From records to a recommendation

1. Read six related CSV files. A CSV is a plain table that can also be opened in a spreadsheet.
2. Check the dates, product IDs, quantities and links between those files.
3. Learn demand patterns from usable past sales and the calendar.
4. Predict the next 30 days; the screen can show 7, 14 or 30 days.
5. Check stock, deliveries and expiry against those predictions.
6. Show draft order quantities and reasons for the owner to review.

If an upload is invalid, the old dataset stays active. The app does not replace only half of the related files.

Which model does the app use?

The current demo uses **Seasonal Poisson Regression**. The Python class is called `PoissonRegressor`, from the scikit-learn library. A separate model is trained for each product that has enough sales history.

A regression model predicts a number. Here, that number is the expected units sold per day. Poisson regression is designed for counts and keeps the expected number above zero. A result such as 7.5 means an average expectation, not that the shop can sell half a packet.

The model learns four kinds of patterns together: changes over the year, differences between weekdays, slow growth or decline over time, and festival preparation days. These produce **one forecast**, which the ordering and expiry pages also use.

The annual pattern uses sine and cosine waves. These are smooth repeating curves that help the model move gradually from one season to the next. The model does not simply apply a rule such as "increase every product in winter."

Festival preparation means the seven days before a festival plus the festival day. For each product, the current model learns one shared festival effect from all past festival windows. It does not learn a separate Diwali effect and Holi effect. We made this choice because there are only two or three past examples of each named festival. Separate effects were reacting too strongly to random variation.

Why choose this model?

It fits the kind of number we want to predict, can learn a different seasonal pattern for each product, and runs on a normal laptop. More importantly, it had the lowest average error in the earlier periods used to choose the model. This is a reason to use it in this demo, not proof that it is the best model for a real shop.

The app also compares it with three other methods. A 28-day average repeats the recent average. A weekly repeat method copies the last week's pattern. HistGradientBoosting is a machine-learning model that combines many small decision trees to learn more complex relationships.

When does the app avoid this model?

The seasonal model needs at least 180 usable daily observations spread across at least 365 days. Otherwise, it uses the 28-day average and labels that choice. If even recent sales history is too limited, it gives no exact forecast. A new product does not get a made-up demand number just because a review asks for it.

Model settings, with their meanings

Setting	Value and meaning
Python library	scikit-learn 1.9.1
Model class	PoissonRegressor; estimates expected daily counts
Separate fit	One model per eligible product
Input columns	12: one time trend, four annual-wave values, six weekday indicators and one festival indicator
Annual waves	Two sine/cosine pairs; describe smooth repeating yearly changes
StandardScaler	Puts input columns on comparable scales before training
alpha	0.03; discourages unnecessarily large fitted effects
max_iter	500; maximum number of training update steps
tol	0.0000001; training stops when changes are small enough
Forecast length	Up to 30 days
Minimum annual history	180 usable observations spanning at least 365 days

These are fixed settings for this experiment. They were not shown to be optimal for every shop. The model does not use customer review counts to increase demand and does not use future sales as an input. It also does not use local weather or a supplied future promotion plan.

The boosting comparison uses 90 training rounds, up to 15 leaves in each tree, a learning rate of 0.08 and a fixed random seed of 42. Its inputs include recent sales averages, older sales values, product ID and calendar information. It is a comparison method, not the currently selected demo model.

How accurate is it?

Real-shop accuracy is not known. The following numbers come from the project's computer-generated data. We should not present them as results from Cynbit customers or real shops.

Method	Error used to choose the model	Error in later test periods
Seasonal Poisson Regression	3.088 units/day	2.860 units/day
HistGradientBoosting	3.178 units/day	2.964 units/day
28-day average	3.507 units/day	3.374 units/day
Repeat the last week	4.230 units/day	4.133 units/day

Both error columns use **MAE: Mean Absolute Error**. It is the average size of a mistake, ignoring whether the forecast was too high or too low. If the actual sale is 10 and the forecast is 7, the error is 3 units. An MAE of 2.860 means the forecast was wrong by about 2.86 units per evaluated product per day on average. It does not mean every forecast is within that amount.

Other result	Value	What it means
WAPE	18.520%	Add all absolute errors, then divide by all actual units sold. Lower is better.
RMSE	3.921 units	An error score that gives extra weight to large mistakes. Lower is better.
Bias	-0.040 units	Average forecast minus actual sales. The small negative value means slight overall underprediction.

Do not call this 81.48% accuracy. Subtracting WAPE from 100 does not create a reliable "percent correct" score. Forecasting predicts quantities, so the size of each error matters.

On the same current dataset, the chosen model's MAE is about 3.5% lower than boosting and 15.2% lower than the average method. That is a comparison of forecast errors, not a percentage of money saved. A three-unit error may be small for a product selling 50 units a day but serious for one selling four.

How were these numbers measured?

First, the models learned from older records and predicted four later 30-day periods. Those periods were used to choose the model. Next, the chosen method was kept fixed while all methods were tested over twelve more 30-day periods, covering 360 days.

At each step, training used only records from before the forecast date. As time moved forward, training could include records that had become available since the previous step. This imitates updating a model as a shop collects more sales.

The main test contains **4,200 product-days**. One product-day means one product on one date. There were 4,320 possible records for the 12 products across 360 days; 120 were left out because sales were limited by stock availability. The thirteenth product has no sales history and is not scored.

This project has been developed and adjusted using the same simulation. It is not an independent real-world test. Earlier versions also used different data, so their scores should not be used to claim a clean before-and-after improvement.

What is in the dataset?

The shop records are **synthetic**, which means a Python program created them. They are not downloaded from a real store and are not Cynbit data. The festival dates are the exception: they were imported from the public Google Calendar called Holidays in India.

Item	Current dataset specification
Store coverage	One fictional kirana store
Sales dates	1 January 2024 to 16 October 2026
History length	1,020 days for each of 12 products
Products	13 listed products; 12 have sales and one is a new product
Sales records	12,240 rows: one product on one day per row
Stock batches	24 rows describing stock held at the latest snapshot
Purchase orders	3 examples of incoming or planned supply
Customer reviews	14 written examples, not real customer messages
Festival dates	28 entries: seven festival names across 2024-2027
Calendar import date	17 October 2026; saved copy, not automatic live sync
Dataset version	winter-calendar-v3
Random seed	42; this makes the same generator repeatable

Products include cola, water, mango drink, milk, yogurt, chips, tea, coffee, biscuits, chocolate, packaged sweets and rice. Sugar-free cola is the new product without sales history. Prices and stock values are examples in Indian rupees.

The generator combines a starting demand level with seasonal changes, weekends, gradual trends, festival periods, example promotions and random variation. It also creates supply interruptions so some days run out of stock. It deliberately includes expiry and overstock examples so those screens can be demonstrated.

The current tea and coffee example has a winter peak. The earlier demo curve peaked near October by mistake; the generator was corrected. That change is an assumption in a fictional example, not something discovered in real shop records. The gifting example also has a festival increase built into the generated demand. A model matching these patterns does not prove that it will match a real shop.

Why this data is a major limitation

Real demand can change with location, prices, promotions, weather, competitors, customer habits and supply problems. The small simulation does not capture all of that. A good-looking test result can therefore coexist with forecasts that feel unrealistic.

We need real sales and stock records, with permission to use them, before judging business usefulness. Those records should include dates, product quantities, price changes, promotions and stock availability. We should test future periods that were not used while adjusting the model, and inspect errors for each product, not only one average score.

Sales are not always the same as demand

Suppose the shop has three milk packets and sells all three. There may have been only three buyers, or there may have been ten. The sales record cannot tell us how many customers left without buying.

When the records show that stock ran out, the app leaves that day out of demand training instead of calling the recorded sale the full demand. Missing dates are also treated as unknown, not as zero sales. This is a simple precaution. It can still miss high-demand days and does not fully solve the problem of lost sales.

The demo includes a special column called simulated_demand: the demand the generator created before stock limits. Real shops usually do not know this number. It is used only for a separate test calculation, never to train the model. The main reported scores use observed sales on days not limited by stock.

How an order quantity is calculated

Lead time is how long a supplier takes to deliver. **Review period** is how many days you wait between checking and placing orders. If delivery takes four days and you review stock every seven days, you plan for eleven days in total.

Safety buffer means extra stock kept as a precaution. A buffer of 1.5 demand-days with expected sales of 20 units per day means 30 extra units. It is a chosen setting, not a guarantee that stock will never run out.

Example: expected sales are 20 a day for 11 days, so planned demand is 220. Add 30 safety units, subtract 80 usable units already held and 20 confirmed incoming units. The starting order

suggestion is 150 units. The actual app adds up each day's forecast, so it can account for changing demand rather than always using a flat 20.

The app then checks expiry, storage space, supplier pack size and minimum order size. Those checks can reduce the quantity. A minimum order quantity, or MOQ, is the smallest amount a supplier will accept. If the safe limit is 20 and packs contain 12, the app can suggest 12 rather than rounding up to an unsafe 24.

What happens to stock that may expire?

The app uses the recorded expiry dates. It does not predict or invent the official expiry date. It estimates how much stock may still be unsold before that date.

It uses **FEFO: First Expiry, First Out**. Each day's expected sales are assigned to the batch that expires first. For example, if 80 yogurt units expire after five days and expected sales are six a day, about 30 may sell and 50 may remain at risk. Two batches cannot both claim the same sales.

Expired stock is not counted as usable. Unknown expiry dates need checking. If expiry is beyond the 30-day forecast, the app cannot estimate the final unsold amount reliably. A blank value therefore means unknown, not safe.

Reviews and calendar data

Reviews are checked using product names and simple English word rules. Phrases can suggest a request, unavailable product, variant or quality problem. This is not an LLM or a trained sentiment classifier. Mixed languages, sarcasm and reviews about several products can confuse it.

Google Calendar supplied a saved set of public festival dates. Personal appointments were not used. The app does not automatically keep that calendar up to date. Dates can be inspected, downloaded or applied in Data Management; local and tentative dates still need review.

Tools used to build the app

Tool	Its job
Python 3.12	Runs the application code
pandas and NumPy	Read and calculate with tables and numbers
scikit-learn	Trains and compares forecasting models
Streamlit	Builds the local web screens
Plotly	Draws interactive charts
SQLite	Saves a dataset locally when the user chooses to save
pytest	Runs checks on code behavior
ReportLab	Turns the written reports into PDFs

What the tests prove, and what they do not

The latest code checks report **26 tests passed**. They cover data checks, stock calculations, expiry, future-data leakage, insufficient history, festival dates, seasonal examples and screen navigation. A passing test means a checked behavior worked in that example. It does not establish real-shop forecast accuracy.

The full forecasting evaluation was also run after the last model change. The numbers are saved in evaluation/metrics.json and the individual predictions in evaluation/backtest_predictions.csv. This language-only report rewrite does not change those results.

Main limitations

The sales are generated, the store and product range are small, and seasonal assumptions are simplified. Festival effects are shared across named festivals for each product. The model can miss sudden changes and does not provide a tested probability range around each forecast. The app has no live Revio connection, no automatic ordering and no production sign-in system.

Reference documents

scikit-learn PoissonRegressor documentation: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.PoissonRegressor.html

scikit-learn HistGradientBoostingRegressor documentation: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html

Forecasting: Principles and Practice, time-based testing: https://otexts.com/fpp3/toolbox.html

These references explain the methods. The project's own code and saved evaluation establish which settings and results were actually used here.